

HLD R1 — Control de acceso a la red según rol

Qué es este documento. Diseño de alto nivel (HLD) del requerimiento R1: *controlar el acceso a la red de usuarios válidos según su rol*. Es **diseño conceptual**, no de software: describe la solución con el detalle suficiente para que un ingeniero competente derive la implementación. El detalle concreto va en el LLSD. Vale **10 puntos** del Ex1. Los números entre corchetes son los pesos de cada criterio de la rúbrica. **Sobre la topología:** este HLD describe la solución para una red de campus genérica. La topología del laboratorio VNRT es *una* realización posible y se usa únicamente como banco de validación (§11); el diseño no depende de ella. **Convención de datos:** (*medido*) = obtenido experimentalmente en el banco de validación, reproducible. (*objetivo*) = compromiso asumido, aún sin verificar. (*analítico*) = cálculo sobre un modelo. Nunca se presenta una estimación como si fuera una medición.

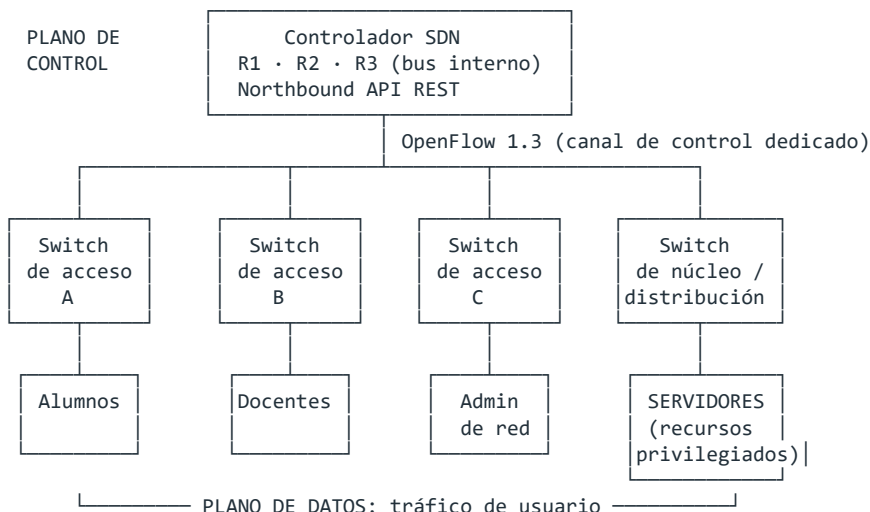
1. Planteamiento del problema

El campus debe permitir que **solo usuarios válidos** accedan a la red, y que cada uno obtenga **exactamente los permisos de su rol**: ni más (riesgo) ni menos (fricción). Tres exigencias que el diseño debe conciliar:

Exigencia	Tensión que genera
El control debe aplicarse a todo el tráfico	Si cada paquete se inspecciona en el controlador, la red no escala
El permiso depende de quién es el usuario, no de dónde se conecta	Las ACL tradicionales atan permisos a IP/puerto, que cambian
Debe reaccionar a eventos de seguridad en tiempo real	Una configuración estática no puede revocar una sesión

La respuesta de este diseño: **decidir una vez en el plano de control, aplicar siempre en el plano de datos**.

2. Modelo de red de referencia



Dos principios de direccionamiento que sostienen todo el diseño:

1. **El rol NO depende de la subred ni de la IP del usuario.** Los usuarios pueden estar en una red de acceso plana; su rol viaja dentro del pipeline del switch, en el campo *metadata*. Esto evita tener que re-direccionar la red por rol y evita que un cambio de IP (DHCP) rompa los permisos.
2. **Los recursos privilegiados sí se agrupan** en una subred de servidores, lo que permite expresar la política por rol × recurso en pocas reglas (ver HLD R2).

3. Lógica de acceso y autenticación de usuarios válidos [16]

Decisión

La identidad del host se establece por su dirección MAC, contrastada contra una tabla de roles administrada desde el controlador. Se declara explícitamente como **decisión de alcance**: es lo suficiente para demostrar y validar la lógica de control de acceso SDN, que es el objeto del requerimiento.

Flujo de identificación

1. El host emite su primer paquete en la red de acceso.
2. El paquete llega a la tabla de identidad del pipeline sin coincidencia y genera un **Packet-In** hacia el controlador.
3. R1 extrae la MAC origen y la busca en la tabla de roles.
4. Si está registrada, obtiene el rol; si no, aplica **rol mínimo por defecto** (alumno) y registra el evento para auditoría.
5. R1 instala una regla que **escribe el rol en metadata** y pasa el paquete a la tabla de política con **goto_table**.
6. Emite el evento **host_autenticado** en el bus interno del controlador.

La objeción previsible, y cómo la responde el diseño

La MAC es **suplantable**. Lo asumimos de frente; el diseño lo compensa por construcción:

Limitación de identificar por MAC	Cómo la compensa la arquitectura
Un atacante puede clonar la MAC de un docente	La primera tabla del pipeline, de R3 , valida la terna (IP origen, MAC origen, puerto físico de ingreso). Una MAC clonada que aparece por un puerto distinto al aprendido se descarta como spoofing antes de llegar a la tabla de identidad
No hay prueba criptográfica de identidad	El alcance del Ex1 es la lógica de control de acceso SDN, no la infraestructura de credenciales. Es un supuesto declarado, no un descuido
No hay caducidad de sesión	R1 mantiene el estado de sesión en metadata y R3 puede forzar sesion_revocada en cualquier momento

Este es el argumento central de la exposición: el par *MAC + puerto de ingreso* es sustancialmente más difícil de falsificar que la MAC sola, y **sale gratis**, porque R3 necesita esa misma validación para detectar IP spoofing. La debilidad de R1 queda cubierta por un módulo que el proyecto tenía que construir de todos modos. Es integración entre requerimientos, no un parche.

Evolución prevista

802.1X con RADIUS es la evolución natural y **no rompe este diseño**: sustituye únicamente el paso 3 (de "buscar MAC en tabla" a "resultado de la autenticación EAP"). Tablas, prioridades, escritura de **metadata** y eventos quedan idénticos. Se documenta como trabajo del Ex2.

4. Definición de usuarios, niveles de acceso y permisos [16]

Cuatro roles, alineados con la codificación de **metadata** del contrato de tablas (**../contratos/tablas-openflow.md**). **Sin superposiciones y sin vacíos**: todo host cae exactamente en un rol, y el rol desconocido degrada a alumno.

Valor	Rol	Nivel	Permisos sobre la red	Recursos accesibles
1	Alumno	Mínimo	Solo consumo; no administra nada	Servicios generales. Denegado en recursos críticos
2	Docente	Medio	Consumo + acceso a evaluación	Generales + servidor de notas

Valor	Rol	Nivel	Permisos sobre la red	Recursos accesibles
3	Administrador de red	Alto	Define políticas vía Northbound API, revoca sesiones, consulta métricas	Plano de gestión y telemetría. Denegado en datos académicos
4	Superusuario	Máximo	Todo lo anterior, sin restricción	Todos, incluido el repositorio de exámenes
0	(desconocido)	—	Degrada a alumno y se registra	Los del alumno

Decisión de diseño deliberada: el administrador de red **no** accede a los datos académicos. Administrar la red y leer notas son funciones distintas; concederle todo al administrador sería un vacío de control. El acceso total existe solo en el superusuario, que es un rol de excepción y auditado. Esto es *separación de funciones*, y conviene decirlo con ese nombre en la exposición.

Matriz rol × recurso (vista de R1; el detalle vive en el HLD de R2)

Recurso	Alumno	Docente	Admin. de red	Superusuario
Servicios generales (portal, DNS, DHCP)	Si	Si	Si	Si
Servidor de notas	No	Si	No	Si
Repositorio de exámenes	No	No	No	Si
Northbound API de gestión	No	No	Si	Si

5. Pertinencia técnica del método de control [10]

Se evalúan cuatro mecanismos y se adopta **RBAC aplicado mediante políticas OpenFlow**, con la identidad resuelta en el controlador.

Mecanismo	Qué aportaría	Por qué no se adopta como base
ACL estáticas en los switches	Simple, sin controlador	No tiene noción de identidad ni de rol: ata permisos a IP/puerto, que cambian. Ingobernable al crecer y no reacciona a eventos de R3
RADIUS / 802.1X	Autenticación fuerte y estándar	Resuelve <i>quién entra</i> , no <i>qué puede alcanzar</i> una vez dentro. Es complementario, no sustituto
ABAC (por atributos)	Políticas muy expresivas (hora, ubicación, postura)	Complejidad de evaluación y prueba desproporcionada para cuatro roles. Se menciona como evolución
RBAC sobre OpenFlow ← adoptado	El rol viaja en metadata dentro del pipeline y se aplica en el switch a velocidad de línea	—

Razones de la elección, en orden de peso:

- El rol se aplica en el plano de datos, no en el controlador.** Solo el primer paquete de cada host sube; el resto se conmuta en el switch. Es lo que hace que la latencia sea de milisegundos y **no crezca con el número de usuarios** (§8).
- Encaja con el mecanismo nativo del protocolo.** **metadata** es exactamente lo que OpenFlow 1.3 ofrece para pasar contexto entre tablas.
- Es reactivo.** Como la decisión vive en el controlador, R1 puede revocar una sesión cuando R3 emite **ataque_detectado**. Una ACL estática no puede.

6. Lógica del proceso desde solicitud hasta autorización [16]



```

T0 (R3): terna
IP+MAC+puerto OK
T1 (R3): sin bloqueo
T2 (R1): SIN COINCIDENCIA

--2. Packet-In----->
--3. buscar MAC---->
<--4. rol = alumno--
<--5. FLOW_MOD-----
    tabla 2, prioridad 20000-29999
    match: eth_src = MAC del host
    acción: write_metadata(rol, sesión)
            goto_table (política)
<--6. confirmación-----
    [evento host_autenticado → R2, R3]

--7. tráfico siguiente> T2 ACIERTA → T3 (política) → T4 (reenvío)
<--8. conmutado en el switch, sin subir al controlador----->

```

Tres puntos para remarcar en la exposición:

- **El costo del control lo paga solo el primer paquete.** Los pasos 2 a 6 ocurren una vez por host. Del paso 7 en adelante, el switch resuelve solo.
 - **El orden de las tablas no es arbitrario.** R3 valida antispoofing *antes* de que R1 asigne identidad, precisamente para que un atacante no obtenga un rol suplantando una MAC. Es la coherencia que responde a la objeción de §3.
 - **Flujo alternativo:** MAC no registrada → rol alumno + evento de auditoría.
- Flujo de excepción:** llega *ataque_detectado* → R1 marca la sesión como revocada y emite *sesion_revocada*.

7. Multiplataforma y portabilidad [10]

Capa	Grado de atadura	Qué haría falta para portarlo
Protocolo	Ninguna. OpenFlow 1.3 es estándar multi-fabricante	Nada
Pipeline de tablas	Requiere ≥5 tablas y soporte de <i>metadata</i>	Los switches de hardware ofrecen menos tablas que un switch por software. Con menos de 5 habría que fusionar tablas conservando el orden relativo
Controlador	Media: el código usa la API de un controlador concreto	Portar a otro controlador implica reescribir los manejadores de eventos, no el diseño . Pipeline, prioridades y eventos se mantienen
Método de identidad	Ninguna	MAC sustituible por 802.1X sin tocar el resto (§3)
Direccionamiento	Alta si se incrustan valores concretos	Parametrizar subredes en configuración, no en el código

Riesgo honesto a declarar: el diseño asume *metadata* y múltiples tablas, ambas de OpenFlow 1.3. Un equipo limitado a OpenFlow 1.0 **no** puede ejecutar este pipeline. No es teórico: en el banco de validación los switches venían configurados en 1.0 y hubo que habilitar 1.3 explícitamente para que el controlador conectara.

8. Escalabilidad: logins por segundo [10]

Una autenticación consume un *Packet-In* y un *FLOW_MOD*: exactamente el ciclo medido en el banco de validación.

Carga ofrecida	Procesado	CPU del controlador	Pérdida
100 /s	100 /s	4 %	0
500 /s	500 /s	17-20 %	0
1 000 /s	1 000 /s	32-34 %	0

Carga ofrecida	Procesado	CPU del controlador	Pérdida
2 000 /s	2 000 /s	47-52 %	0
4 000 /s	4 000 /s	53-67 %	0
saturación	~11 000 /s (techo)	~110 % (núcleo saturado)	masiva

(medido)

Conclusiones:

- **Techo duro: ~11 000 autenticaciones/s.** Por encima, el controlador pierde eventos.

- **Techo operativo recomendado: ~4 000/s**, que es donde la CPU se mantiene en torno al 60 % comprometido.

- **Dimensionamiento:** un campus de 10 000 usuarios conectándose en una hora pico repartida en 10 minutos genera del orden de **17 autenticaciones/s de media**. El margen frente al techo operativo es de **más de dos órdenes de magnitud**.

Dónde está el cuello de botella, con evidencia: en el **plano de control**, y concretamente en que el controlador procesa eventos en un solo hilo. En saturación un núcleo llega al 100 % mientras los demás quedan ociosos. La vía de crecimiento no es una máquina más grande: es **repartir switches entre varias instancias de controlador**. El plano de datos no es el límite — instalar reglas costó ~140 µs y el switch aceptó 50 000 reglas sin rechazar ninguna (medido).

9. Latencia con múltiples usuarios [11]

Latencia del ciclo **Packet-In** → regla confirmada, que es la latencia de autenticación que percibe el usuario en su primer paquete.

Carga concurrente	Mediana	p95	¿Cumple el objetivo de 500 ms?
100 /s	1.45 ms	1.45 ms	Sí, con ~344× de margen
1 000 /s	~0.95 ms	~1.5 ms	Sí
4 000 /s	~0.87 ms	~1.6 ms	Sí
saturación (~11 000 /s)	~0.9-1.2 ms	175 ms → 4 100 ms	No

(medido)

Tres lecturas:

1. **La latencia no se degrada con la concurrencia** mientras el controlador no

sature: entre 100 y 4 000 autenticaciones/s se mantiene por debajo de 2 ms. La razón es de diseño: solo el primer paquete sube al controlador.

2. **La degradación no es gradual, es un acantilado.** Al saturar, la mediana sigue

baja pero el **p95 se dispara a segundos**, porque la cola de eventos se acumula. Por eso el indicador a vigilar es el **p95, no el promedio**: un promedio sano puede ocultar que un 5 % de los usuarios espera 4 segundos.

3. El usuario percibe esta latencia **una sola vez**, al conectarse.

10. Supervisión del correcto funcionamiento [11]

Qué se registra

Evento	Cuándo	Para qué sirve
host_autenticado	Cada asignación de rol	Auditoría: quién entró, con qué rol, por qué puerto

Evento	Cuándo	Para qué sirve
MAC desconocida	Al aplicar rol mínimo por defecto	Detectar equipos no inventariados
<code>sesion_revocada</code>	R3 confirma un ataque	Trazabilidad de la respuesta automática
Rechazo por antispoofing	Terna IP/MAC/puerto inconsistente	Señal temprana de suplantación

Cómo se expone (Northbound API REST)

Endpoint	Devuelve
<code>GET /r1/sesiones</code>	Hosts autenticados: MAC, IP, rol, switch, puerto, antigüedad
<code>GET /r1/metricas</code>	Autenticaciones/s, latencia mediana y p95, CPU, sesiones activas
<code>GET /r1/roles</code>	Tabla de roles vigente
<code>POST /r1/revocar</code>	Revocación manual por el administrador de red

Verificación continua

El indicador de salud del módulo es la **coherencia entre el estado del controlador y el plano de datos**: para cada sesión activa debe existir su regla en la tabla de identidad, dentro del rango de prioridad 20000-29999. Una sesión sin regla, o una regla fuera de rango, es una alarma. Se verifica automáticamente en `tests/integration/`.

11. Validación del diseño en banco de laboratorio

Las cifras de §8 y §9 no son estimaciones: provienen de un banco ejecutado sobre el entorno VNRT, con controlador real y switches OpenFlow 1.3 reales.

Aspecto validado	Resultado
Soporte de <code>write_metadata</code> + <code>goto_table</code>	Confirmado — el pipeline es implementable tal como se diseñó
Número de tablas disponibles	254 por switch; el pipeline de 5 tablas cabe holgadamente
Capacidad del plano de control	~11 000 eventos/s de techo; ~4 000/s dentro del presupuesto de CPU
Latencia de autenticación	1.45 ms a 100 autenticaciones/s
Capacidad de instalación de reglas	50 000 reglas sin rechazo, ~140 µs por regla

La topología concreta del banco es incidental: lo que se valida es que **el mecanismo** (identidad → `metadata` → política → reenvío) funciona y con qué márgenes.

12. Interfaz con los otros módulos

R1 escribe el rol en `metadata` en la **tabla de identidad** y usa prioridades **20000-29999**. Ver `../contratos/tablas-openflow.md`.

```

R1 --host_autenticado--> R2 (conoce el rol para aplicar política)
R1 --host_autenticado--> R3 (línea base de la terna IP/MAC/puerto)
R3 --ataque_detectado--> R1 (revoca sesión)
R1 --sesion_revocada---> R2 (extiende el bloqueo)

```

13. Métricas comprometidas

Métrica	Objetivo	Estado	Cómo se mide
Latencia de autenticación (p95)	≤ 500 ms	1.45 ms a 100/s (<i>medido</i>)	Banco de plano de control
CPU del controlador a 100 flujos/s	≤ 60 %	4 % (<i>medido</i>)	Instrumentación del proceso
Logins concurrentes por segundo	—	~4 000/s operativo · ~11 000/s techo (<i>medido</i>)	Rampa de carga

Métrica	Objetivo	Estado	Cómo se mide
Roles jerárquicos soportados	4	4 (<i>diseño</i>)	Codificación de <i>metadata</i>
Tasa de bloqueo de accesos inválidos	≥ 95 % (<i>objetivo</i>)	Pendiente de medir	<i>tests/integration/</i> , banco de ataques

Declaración honesta: la tasa de bloqueo de accesos inválidos **todavía no está medida**; requiere el banco de ataques, aún no ejecutado. No se presenta ningún número en su lugar. Está diseñado y el procedimiento de medición está definido; la medición es el siguiente paso.